



Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3067 – Redes

Laboratorio 1

Esquemas de comunicación e introducción a Wireshark

Reporte Individual — Segunda Parte

Dilary Sarahí Cruz López

Carné: 231010

Guatemala, 13 de julio de 2026

1. Introducción

El presente reporte documenta la segunda parte del Laboratorio 1 del curso de Redes, enfocada en el uso del analizador de paquetes Wireshark. Un analizador de paquetes permite capturar el tráfico que circula por una interfaz de red y examinar en detalle cada transmisión: direcciones de origen y destino, protocolos utilizados, puertos y el contenido de los mensajes. Esto resulta fundamental para comprender de forma práctica cómo viajan realmente los datos por la red, así como para diagnosticar cuellos de botella o posibles vulnerabilidades de seguridad.

El trabajo se dividió en tres actividades: la personalización del entorno de Wireshark (creación de un perfil propio, columnas, reglas de color y filtros), la configuración de una captura con ring buffer, y el análisis de una comunicación real bajo el protocolo HTTP. A continuación se presentan las capturas de pantalla que evidencian cada paso, junto con las respuestas a las preguntas planteadas, la discusión de la experiencia y las conclusiones.

2. Personalización del entorno (Sección 3.4)

Se inició Wireshark y se creó un perfil de configuración con el nombre y apellido de la estudiante ("Dilary Cruz"), el cual se mantiene activo durante toda la práctica y es visible en la barra de estado inferior. Sobre el archivo intro-wireshark-trace1.pcap proporcionado se aplicaron las siguientes personalizaciones.

2.1 Perfil personalizado y formato de tiempo

Se aplicó el formato de tiempo "Time of Day", por lo que la columna Time muestra la hora real del día (por ejemplo 13:22:45.438793) en lugar de los segundos transcurridos desde el inicio de la captura. En la esquina inferior derecha se observa el perfil activo "Profile: Dilary Cruz".

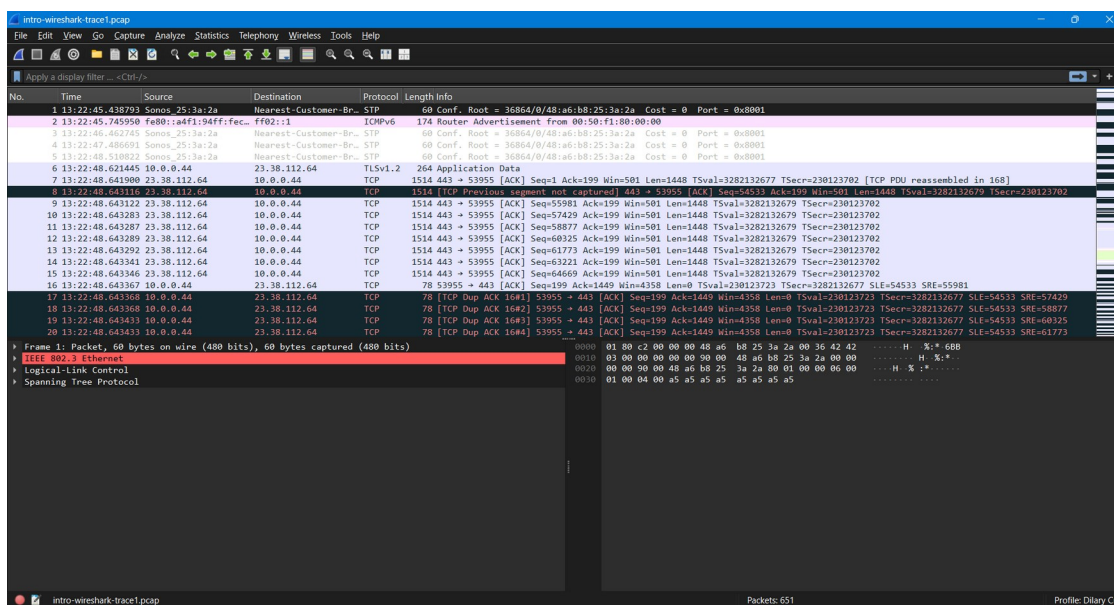


Figura 1. Perfil personalizado "Dilary Cruz" y formato de tiempo Time of Day aplicado.

2.2 Columna de longitud del protocolo

Se agregó una columna personalizada de tipo Custom con el campo frame.len, que muestra la longitud de cada paquete. Adicionalmente, se ocultó la columna "Length" que trae Wireshark por defecto, dejando visible únicamente la columna personalizada "Protocol Length" (visible al extremo derecho de la lista de paquetes).

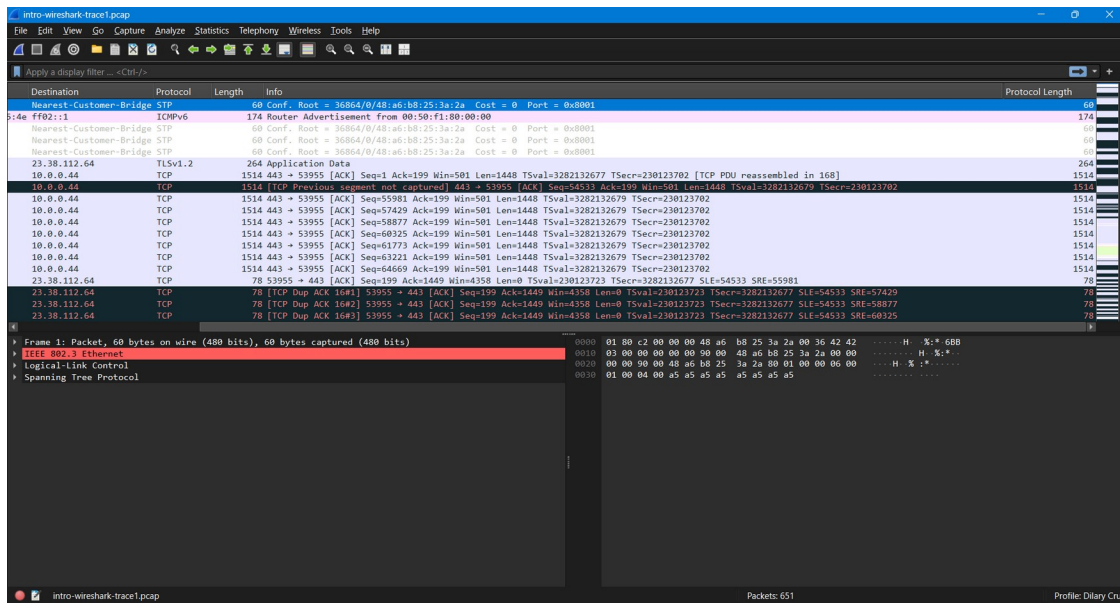


Figura 2. Columna personalizada "Protocol Length" (frame.len) agregada y columna Length por defecto oculta.

2.3 Distribución de paneles (Layout)

Se modificó la distribución de los paneles a un esquema distinto al predeterminado. En lugar de tener los tres paneles apilados verticalmente, se ubicó la lista de paquetes a la izquierda y los paneles de detalle y de bytes hexadecimales lado a lado a la derecha.

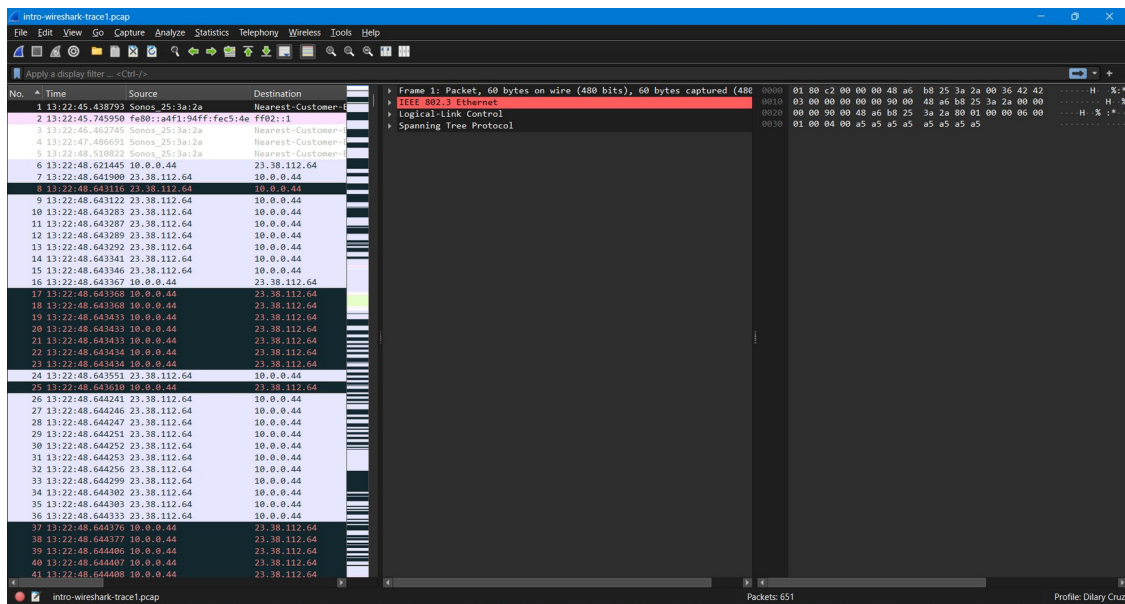


Figura 3. Distribución de paneles personalizada, diferente al esquema por defecto.

2.4 Regla de color y botón de filtro

Se creó una regla de color llamada "TCP SYN" con el filtro tcp.flags.syn == 1, ubicada al inicio de la lista de reglas para que se aplique primero, con un color de fondo verde. También se creó un botón de acceso rápido llamado "SYN" que aplica ese mismo filtro. En la siguiente captura se observa el botón activo y los paquetes con la bandera SYN resaltados en verde.

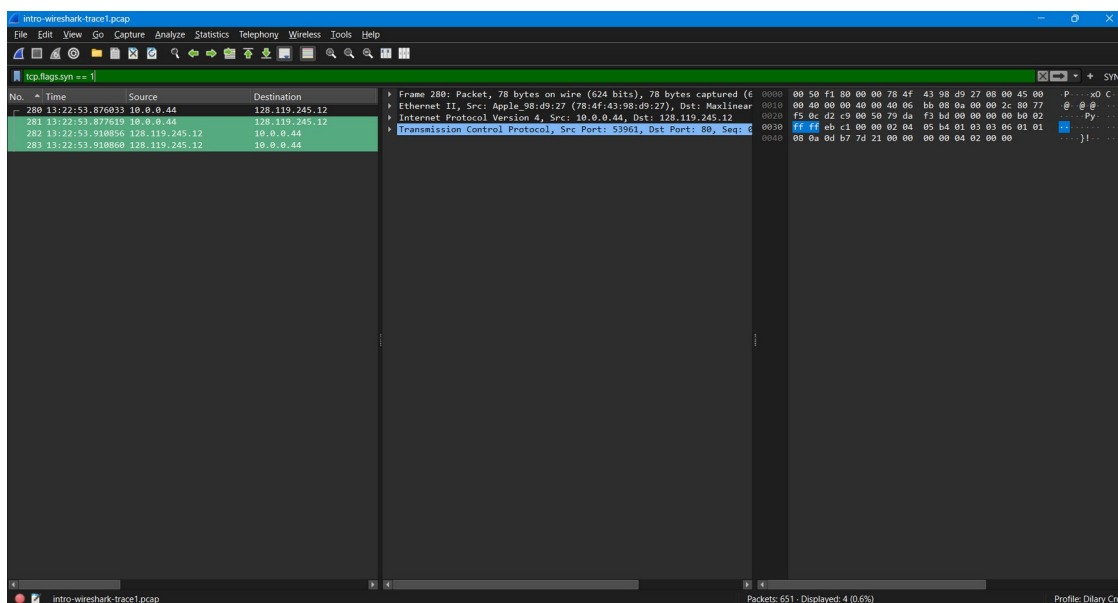


Figura 4. Regla de color TCP SYN=1 (paquetes en verde) y botón de filtro "SYN" en funcionamiento.

2.5 Lista simplificada de interfaces

Se ocultaron las interfaces virtuales del sistema mediante la opción Manage Interfaces, dejando visible únicamente la interfaz física real de red inalámbrica "Intel(R) Wireless-AC 9462: Wi-Fi". De esta forma la lista de interfaces de captura queda simplificada.

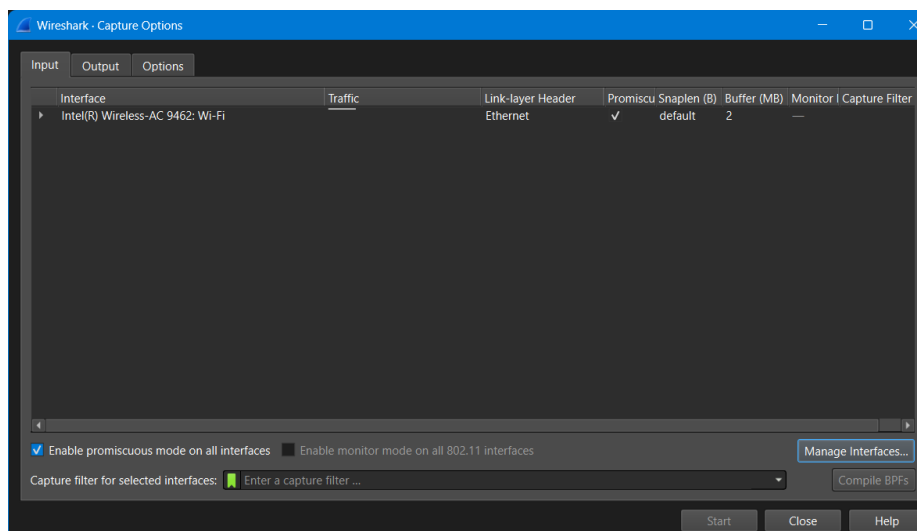


Figura 5. Lista simplificada de interfaces: solo se muestra la interfaz Wi-Fi real.

3. Configuración de la captura de paquetes (Sección 3.5)

3.1 Comando ipconfig

Se ejecutó el comando ipconfig en la terminal de Windows para observar la configuración de red del equipo. La salida muestra dos adaptadores "Wireless LAN adapter Local Area Connection* 1 y 2", que corresponden a adaptadores virtuales de Windows (Wi-Fi Direct) y aparecen desconectados ("Media disconnected"). Estos coinciden con las interfaces virtuales que se ocultaron en Wireshark. El adaptador real y activo es "Wireless LAN adapter Wi-Fi", con la siguiente configuración:

- Dirección IPv4: 10.100.17.45
- Máscara de subred: 255.255.224.0
- Puerta de enlace predeterminada (Gateway): 10.100.0.1
- Dirección IPv6 de enlace local: fe80::ba91:3b57:932d:7e08%21

Este comando confirma la existencia de los adaptadores virtuales fantasma que Windows crea automáticamente, y que no corresponden a interfaces físicas reales.

```
PS C:\Users\dilar> ipconfig

Windows IP Configuration

Wireless LAN adapter Local Area Connection* 1:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::ba91:3b57:932d:7e08%21
    IPv4 Address. . . . . : 10.100.17.45
    Subnet Mask . . . . . : 255.255.224.0
    Default Gateway . . . . . : 10.100.0.1
PS C:\Users\dilar>
```

Figura 6. Salida del comando ipconfig mostrando la interfaz Wi-Fi real y los adaptadores virtuales.

3.2 Captura con ring buffer

Se configuró una captura de paquetes sobre la interfaz Wi-Fi utilizando un ring buffer. En la pestaña Output se estableció el nombre de archivo lab1_231010, un tamaño máximo de 5 megabytes por archivo (opción "Create a new file automatically after 5 megabytes") y un máximo de 10 archivos (opción "Use a ring buffer with 10 files"). Con el ring buffer, Wireshark conserva únicamente los últimos archivos generados y va sobrescribiendo los más antiguos, lo que evita llenar el disco durante capturas prolongadas.

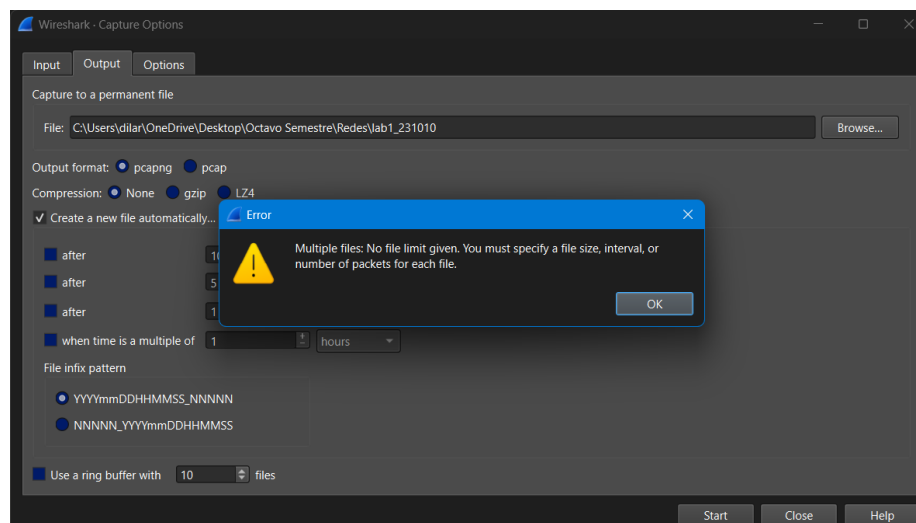


Figura 7. Configuración del ring buffer: nombre lab1_231010, 5 MB por archivo y 10 archivos.

Tras iniciar la captura se generó tráfico navegando por Internet, lo que provocó la creación automática de múltiples archivos de aproximadamente 5 MB cada uno, numerados de forma consecutiva (lab1_231010_00001, lab1_231010_00002, etc.).

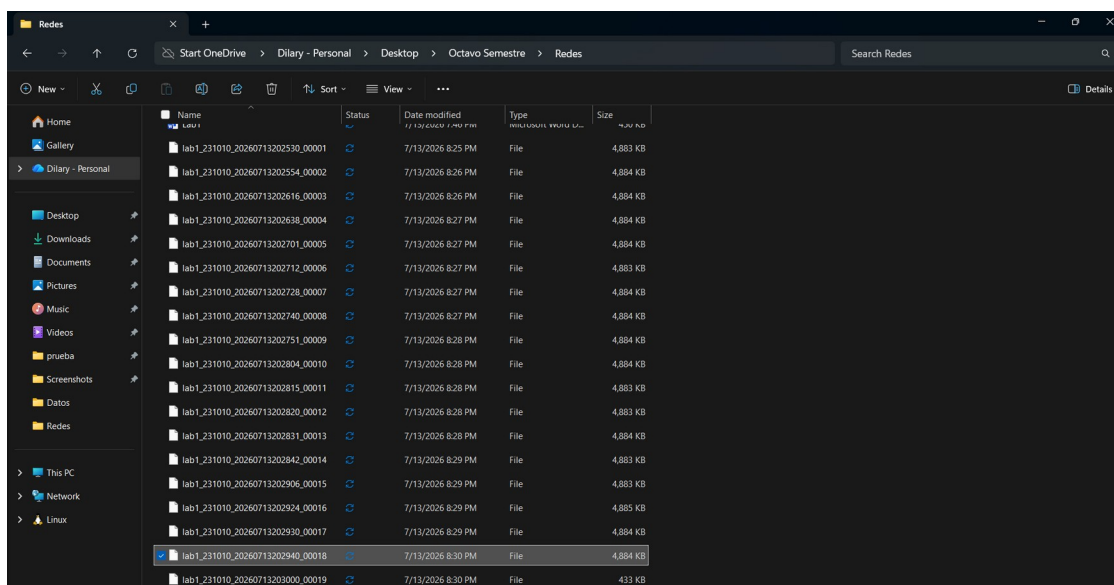


Figura 8. Archivos generados por el ring buffer, de ~5 MB cada uno, en la carpeta del proyecto.

4. Análisis de paquetes HTTP (Sección 3.6)

Se inició una captura sin filtro sobre la interfaz Wi-Fi y se accedió desde el navegador a la dirección <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>. Luego de detener la captura, se aplicó el filtro de visualización http para aislar los paquetes del protocolo HTTP. Se identificaron dos paquetes relevantes: la solicitud GET enviada por el navegador al servidor y la respuesta 200 OK enviada por el servidor.

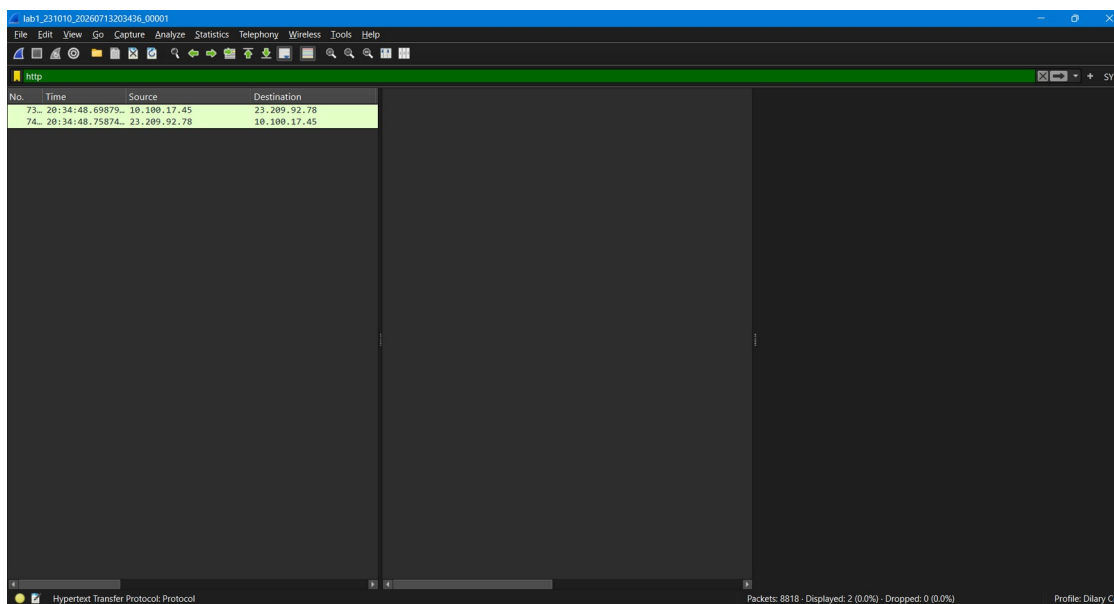


Figura 9. Paquetes HTTP filtrados: solicitud GET y respuesta 200 OK.

4.1 Solicitud del navegador (GET)

Al expandir el paquete de solicitud se observa la petición GET dirigida a `gaia.cs.umass.edu`, con la versión del protocolo, el User-Agent del navegador y las cabeceras Accept y Accept-Language.

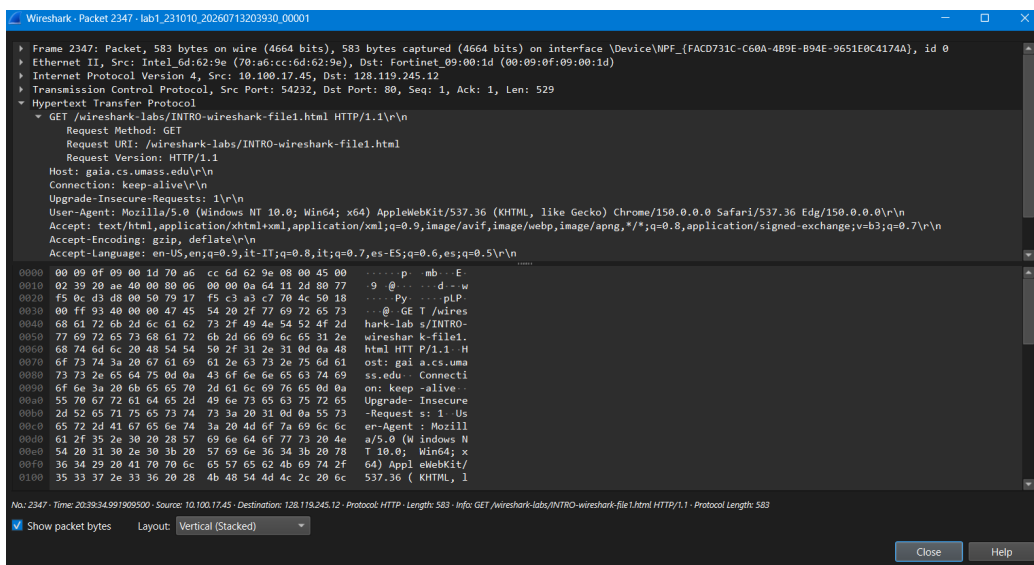


Figura 10. Detalle de la solicitud GET del navegador hacia `gaia.cs.umass.edu`.

4.2 Respuesta del servidor (200 OK)

Al expandir el paquete de respuesta se observa el código de estado 200 OK, la versión del protocolo del servidor, el software del servidor (Apache) y la cabecera Content-Length con la cantidad de bytes de contenido devueltos.

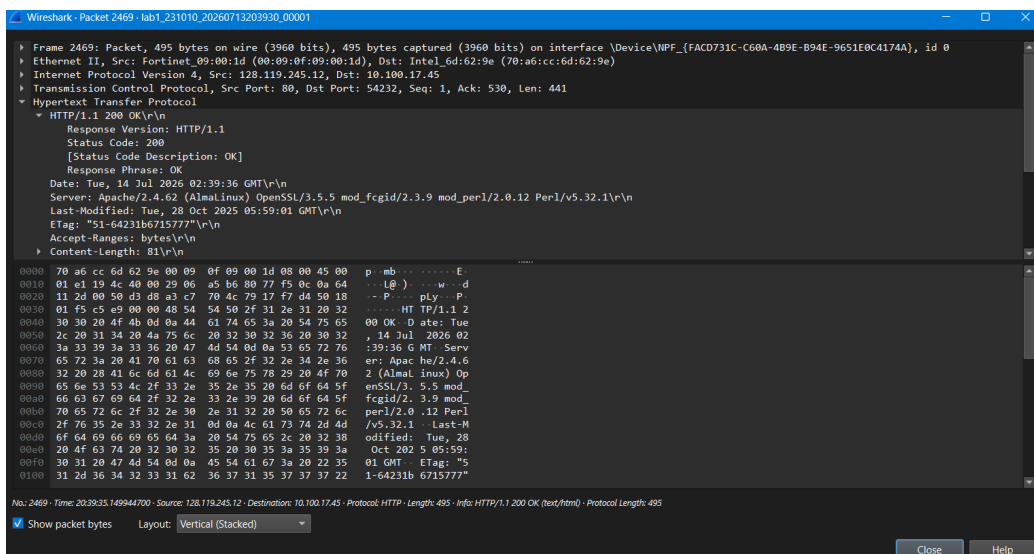


Figura 11. Detalle de la respuesta 200 OK del servidor.

4.3 Respuestas a las preguntas

a) ¿Qué versión de HTTP está ejecutando su navegador? El navegador utiliza HTTP/1.1, como se indica en la línea "Request Version: HTTP/1.1" de la solicitud GET.

b) ¿Qué versión de HTTP está ejecutando el servidor? El servidor también responde con HTTP/1.1, como se indica en la línea "Response Version: HTTP/1.1" de la respuesta.

c) ¿Qué lenguajes indica el navegador que acepta al servidor? Según la cabecera Accept-Language, el navegador acepta: en-US, en, it-IT, it, es-ES y es (inglés de Estados Unidos, inglés, italiano de Italia, italiano, español de España y español).

d) ¿Cuántos bytes de contenido fueron devueltos por el servidor? El servidor devolvió 81 bytes de contenido, según la cabecera Content-Length: 81 de la respuesta.

e) En caso de un problema de rendimiento al descargar la página, ¿en qué elementos de la red convendría "escuchar" los paquetes? ¿Es conveniente instalar Wireshark en el servidor?

Convendría capturar el tráfico en distintos puntos del recorrido de los paquetes para localizar dónde se origina el problema: en el propio equipo cliente (para descartar problemas locales de la interfaz o del navegador), en el router o puerta de enlace de la red local, y en los enlaces intermedios hacia el servidor. Comparar las capturas en estos puntos permite identificar en qué tramo se produce el retraso o la pérdida de paquetes.

En cuanto a instalar Wireshark en el servidor, en general no es conveniente ni práctico. Por un lado, no se suele tener acceso administrativo a servidores de terceros como gaia.cs.umass.edu, lo cual lo hace inviable. Por otro lado, aunque se tuviera acceso, instalar y ejecutar un analizador de paquetes en un servidor de producción consume recursos y puede afectar su rendimiento, además de implicar riesgos de seguridad. Lo recomendable es capturar en el cliente y en los puntos intermedios de la red bajo nuestro control, que es donde realmente podemos actuar.

5. Discusión, hallazgos y comentarios

La práctica permitió aplicar de forma concreta los conceptos teóricos vistos en clase sobre cómo viajan los paquetes por la red. Durante la primera parte del laboratorio (esquemas de comunicación con código Morse y Baudot), se experimentó de manera manual la dificultad de codificar y decodificar información y la aparición de errores en la transmisión. Esta segunda parte, en cambio, mostró cómo una herramienta profesional como Wireshark automatiza la captura y el análisis de ese mismo tipo de comunicación, pero a la escala real de las redes modernas.

Uno de los aspectos más interesantes fue el análisis del comportamiento del navegador. Al inspeccionar la solicitud GET se pudo ver información que normalmente pasa desapercibida, como el User-Agent, la versión del protocolo y la lista de idiomas que el navegador declara aceptar. Resultó llamativo comprobar que el navegador anuncia varios idiomas ordenados por preferencia, y que toda esta información viaja en texto plano dentro de una petición HTTP.

Otro hallazgo importante fue el papel de la caché del navegador. Al intentar capturar la página por primera vez, si el navegador ya tenía almacenado el contenido, no se generaba tráfico HTTP hacia el servidor y por lo tanto no aparecían los paquetes esperados. Fue necesario comprender que, para observar la comunicación completa, debe forzarse una nueva solicitud al servidor (por ejemplo limpiando la caché). Esto evidencia cómo los mecanismos de optimización, como la caché, modifican el tráfico real que se observa en la red.

También se comprendió la utilidad práctica de funciones como las reglas de color, los botones de filtro y las columnas personalizadas, que facilitan enormemente la lectura de grandes volúmenes de paquetes, así como el mecanismo del ring buffer para gestionar capturas prolongadas sin saturar el almacenamiento.

La principal dificultad encontrada fue distinguir el tráfico relevante del laboratorio (la petición a gaia.cs.umass.edu) del tráfico de fondo generado automáticamente por el sistema operativo, como las

verificaciones de conexión de Windows hacia msftconnecttest.com. Esto reforzó la importancia de saber filtrar e interpretar correctamente los paquetes.

6. Conclusiones

- Wireshark es una herramienta poderosa que permite observar de forma detallada y real cómo viajan los paquetes por la red, reforzando de manera práctica los conceptos teóricos sobre protocolos, direcciones y puertos.
- La personalización del entorno (perfiles, columnas, reglas de color, botones de filtro y distribución de paneles) mejora significativamente la eficiencia al analizar grandes cantidades de tráfico.
- El análisis del protocolo HTTP mostró que gran parte de la información de una comunicación web (versión del protocolo, idiomas aceptados, software del servidor y tamaño del contenido) es accesible directamente desde la captura, y que HTTP transmite estos datos en texto plano.
- La caché del navegador influye directamente en el tráfico que se observa: sin forzar una nueva solicitud, la comunicación con el servidor puede no capturarse, lo que hay que tener en cuenta al analizar tráfico HTTP.
- Para diagnosticar problemas de rendimiento conviene capturar en el cliente y en los puntos intermedios de la red bajo nuestro control, y no en el servidor, ya que normalmente no se tiene acceso a él y su análisis podría afectar su funcionamiento.

7. Referencias

Universidad del Valle de Guatemala. (2026). Laboratorio 1 – Esquemas de comunicación e introducción a Wireshark. Departamento de Ciencias de la Computación, CC3067 Redes.

Wireshark Foundation. (s. f.). Wireshark User's Guide. Recuperado de <https://www.wireshark.org/docs/>

Kurose, J. F. & Ross, K. W. Wireshark Labs: Getting Started (gaia.cs.umass.edu/wireshark-labs).

Microsoft. (s. f.). ipconfig – Documentación de comandos de Windows. Microsoft Learn.